Dynamic Traffic-Aware Network Routing and Self-Healing System using Dijkstra's Algorithm

Sudeep Sharma
Dept. of Electronics and Communication
Manipal Institute of Technology, Manipal
sudeep.mitmpl2023@learner.manipal.edu
Reg. no 230907340

Prakhar Singh
Dept. of Electronics and Communication
Manipal Institute of Technology, Manipal
prakhar5.mitmpl2023@learner.manipal.edu
Reg. no 230907450

***Abstract*—Abstract—In this paper, a simulation of a dynamic communication network using Dijkstra's algorithm for routing has been created on the MATLAB platform. This approach differs from existing static routing methods in that it dynamically responds to network congestion or failures by updating network weights. The system has been designed with a self-healing feature that automatically re-calculates routing paths upon failures. A dual metric approach using link cost and bottleneck bandwidth has been introduced to make this simulation more realistic. A pseudocode specification of this routing engine has been developed, along with some details on how this has been implemented on the MATLAB platform. A comparison with static routing has been performed to show that this approach is indeed a practical solution. The results show that this approach takes less than a millisecond to converge on average, at 0.54 ms.**

***Index Terms*—Dijkstra Algorithm, Dynamic Routing, Self-Healing Networks, MATLAB, Congestion Control, Bottleneck Bandwidth, Network Resilience.**

I. Introduction

The modern landscape of communication has witnessed a dynamic evolution with the emergence of internet-enabled gadgets and devices, cloud computing services, and IoT-based applications. The number of interconnected IoT devices is expected to exceed 29 billion by 2030. This has created unprecedented demands on the existing network infrastructure. Static routing techniques define routes at configuration time; they do not consider any specific conditions at run time. This leads to inefficient bandwidth utilization, increased latency, and potential single-point failures. Static routing techniques cannot respond to unexpected traffic surges and hardware failures without human intervention. This response time is in terms of minutes and hours rather than milliseconds. Other standard routing protocols such as OSPF rely on dead interval timers set to 40 seconds to announce failed links and compute alternative paths. With the dynamic evolution of network traffic patterns and their unpredictable nature, such limitations have become a significant barrier to reliable and high-availability communication. In light of these challenges, this paper proposes a traffic-aware dynamic routing system based on MATLAB and Dijkstra's Algorithm. This system monitors the network conditions and computes optimal paths in real time to ensure efficient and uninterrupted communication.

The novelty in this paper is mainly in the integration of interactive stress testing, dual-metric link evaluation based on cost and bandwidth, and the autonomous self-healing capability, which are not present in traditional static routing approaches. The system has a convergence time averaging at 0.54 ms for a 25-node crash scenario.

Authors' Main Contributions and Novelty:

- **What is existing:** Static routing protocols (RIP, OSPF) with fixed topology assumptions and no runtime adaptability.
- **Our contribution:** A MATLAB-based dynamic routing simulator with randomized topology, double-metric evaluation, interactive failure injection, and autonomous self-healing, combined in a single tool.
- **Novelty:** The combination of real-time weight modification, sub-millisecond re-convergence, and bottleneck bandwidth reporting in an accessible simulation environment has not been previously demonstrated on the MATLAB platform.
- **Improvement over existing results:** The <1 ms convergence time is an improvement over OSPF's typical 40-second dead-interval timer by over four orders of magnitude.

II. Literature Review

Static routing protocols such as RIP (Routing Information Protocol) and OSPF (Open Shortest Path First) have been at the core of network communication. While RIP uses hop count as the only criteria for routing, it is limited to a maximum of 15 hop networks. OSPF is more scalable; however, it uses Link State Advertisements (LSAs) that result in convergence times of tens of seconds after a topology change.

Dijkstra's shortest path algorithm is a more efficient $O\big((V+E)\log V\big)$ method for computing minimum-cost paths in a weighted graph. The algorithm was originally proposed in 1959; however, the classical algorithm does not accommodate changes to the weight of the edges during runtime.

Recent developments in Software-Defined Networking (SDN) and virtualization of network functions have rekindled interest in dynamic centralized routing control. While OpenFlow-based SDN controllers such as ONOS and OpenDaylight provide for dynamic path computation, they also result in controller overhead and single-point-of-failure risks. Other research efforts have focused on machine learning

to predict congestion using shortest path algorithms; however, these require large training datasets and computational overheads that may not be feasible for resource-constrained environments.

Simulation studies on network routing have largely employed either the NS-3 or the GNS-3 simulation environment [5]. While they provide high protocol fidelity, they have poor learning curves and lack interactive visualization. MATLAB, on the other hand, is known to have a rapid prototyping environment with inbuilt support for graph processing and real-time visualization, making it suitable for network simulation.

This work is an extension of Dijkstra's classic algorithm, incorporating dynamic weight changes, dual metric evaluation, and an interactive failure simulation interface, which is missing in existing network visualization.

III. System Model

A. Network Graph Model

The communication network is modeled as a weighted undirected graph:

$$G = (V,\ E,\ W,\ B)$$

where V is the set of routers (nodes), $E \subseteq V \times V$ is the set of communication links (edges), $W : E \to \mathbb{R}^+$ is the cost function, and $B : E \to \mathbb{R}^+$ is the bandwidth function.

The path cost between source s and destination t via path P is:

$$C(P) = \sum_{(u,v)\,\in\,P} w(u,v)$$

The bottleneck bandwidth of path P is the minimum-capacity link along the path, i.e., the factor that limits end-to-end throughput:

$$\mathrm{BW}_{\mathrm{bottleneck}}(P) = \min_{(u,v)\,\in\,P} b(u,v)$$

Dynamic routing minimizes $C(P)$. The self-healing mechanism guarantees that $C(P) < \infty$ whenever the graph remains connected, i.e., a valid path always exists.

B. Failure Models

Two failure classes are formally defined and modeled in the cost matrix:

- **Link Failure:** $w(u,v) \leftarrow \infty$, effectively removing edge (u,v) from G.
- **Node Failure:** $\forall\ v' \in \mathcal{N}(v) :\ w(v,v') \leftarrow \infty$, removing all edges incident to failed node v, where $\mathcal{N}(v)$ is the neighbor set of v.

IV. Methodology

The system is designed and implemented across five distinct phases. Table I summarises each phase, its purpose, and the corresponding MATLAB module.

TABLE I
Five-Phase System Architecture

#	Phase	Description / MATLAB Module
1	Topology Generator	Spanning tree + random mesh; `DijkstraSimMain.m`
2	Routing Engine	Sorted-PQ Dijkstra; `customDijkstraPQ.m`
3	Congestion Sim.	Weight injection + re-convergence; `DijkstraSimMain.m` (Option 1)
4	Failure Recovery	Link/node removal + self-healing; `DijkstraSimMain.m` (Options 2,4)
5	Performance Eval.	`tic`/`toc` timer + BW analysis; `DijkstraSimMain.m`

A. Phase 1: Dynamic Network Foundation

The system moves away from fixed topologies through a *Randomized Topology Generator*:

1) **Node Specification:** The user defines the number of routers N at runtime.
2) **Connectivity Guarantee:** A base spanning tree is constructed over all N nodes, ensuring the network is connected at initialization with exactly $N-1$ edges.
3) **Random Mesh Creation:** Each non-tree node pair is connected with probability $p = 0.30$, adding cross-links that simulate real-world internet mesh topology and create redundant paths.
4) **Dual-Metric Initialization:** Each link (u,v) is assigned a *Cost* $w(u,v) \in [1,50]$ (simulating latency/distance) and a *Bandwidth* $b(u,v) \in \{10,100,1000\}$ Mbps (simulating physical link capacity).

B. Phase 2: Routing Engine (Dijkstra's Algorithm)

A min-priority-queue-optimized implementation of Dijkstra's Algorithm serves as the routing engine. The formal pseudocode is given in Algorithm 1.

Algorithm 1 `customDijkstraPQ`: Priority-Queue Dijkstra Routing

Require: Adjacency matrix `adjMatrix`, `startNode` s, `endNode` t

Ensure: Optimal path `optimalPath` P^*, total cost `totalCost` C^*

1: $distances[v] \leftarrow \infty$ **for all** v; $distances[s] \leftarrow 0$
2: $previous[v] \leftarrow 0$ **for all** v
3: Initialise priority queue $pq \leftarrow \{(0,\ s)\}$
4: **while** pq is not empty **do**
5: Sort pq by distance (ascending); extract front entry
6: $currentDist \leftarrow pq[1,1]$; $u \leftarrow pq[1,2]$; remove row 1 from pq
7: **if** $currentDist > distances[u]$ **then**
8: **continue** {Stale entry; skip}
9: **end if**
10: **if** $u = t$ **then**
11: **break**
12: **end if**
13: **for** each node $v = 1$ **to** N **do**
14: $weight \leftarrow$ `adjMatrix`$[u, v]$
15: **if** $weight > 0$ **and** $weight \neq \infty$ **then**
16: $alt \leftarrow distances[u] + weight$
17: **if** $alt < distances[v]$ **then**
18: $distances[v] \leftarrow alt$; $previous[v] \leftarrow u$
19: Append $(alt,\ v)$ to pq
20: **end if**
21: **end if**
22: **end for**
23: **end while**
24: $C^* \leftarrow distances[t]$
25: **if** $C^* = \infty$ **then**
26: **return** $\emptyset,\ \infty$
27: **end if**
28: Reconstruct P^*: backtrack via $previous[\,]$ from t to s
29: **return** $P^*,\ C^*$

The priority queue is implemented as a sorted matrix that is re-sorted after each insertion using `sortrows`, making each iteration $O(k \log k)$ where k is the current queue length. Stale entries (line 7) are discarded without reprocessing, preserving correctness. For the tested 10–25 node networks, the total runtime is well under 1 ms on standard hardware.

C. Phase 3: Interactive Stress Testing (Congestion)

Congestion is modeled by *weight injection*: the cost $w(u, v)$ of a selected link is increased by a user-defined factor (e.g., from 10 to 200), simulating heavy traffic load. Upon this change, the routing engine triggers immediate re-convergence. The system demonstrates adaptive intelligence by abandoning a direct but congested path in favour of a longer, lower-total-cost detour.

The interactive simulation menu provides five options for stress testing:

- **Option 1 – Traffic Congestion:** Increases link cost to simulate heavy traffic.
- **Option 2 – Link Failure:** Assigns infinite cost to simulate a severed physical cable.
- **Option 3 – Change Source/Destination:** Tests different source-destination routing scenarios.
- **Option 4 – Node Failure:** Simulates a complete router crash and triggers self-healing rerouting.
- **Option 0 – Exit:** Terminates the simulation cleanly.

D. Phase 4: Self-Healing and Failure Recovery

The autonomous recovery sequence triggered upon any network state change is as follows:

1) A network state change is detected (weight update or node removal).
2) The cost matrix is updated: the affected row/column entries are set to ∞.
3) Dijkstra's algorithm is re-executed from the current source.
4) If a valid path exists ($C^* < \infty$), the routing table is updated and the new path is highlighted in green.
5) If no valid path exists, a network-partition alert is raised and the administrator is notified.

Two hardware failure classes are handled:

- **Link Failure:** $w(u, v) \leftarrow \infty$, removing the link from the active graph.
- **Node Failure (Router Crash):** All cost matrix entries for the failed node are set to ∞. The node is rendered in red with red dashed lines on its former connections.

E. Phase 5: Visualization

The network state is conveyed through a real-time, color-coded MATLAB plot updated after every routing event:

- **Green lines:** Current optimal path.
- **Red / Red Dashed:** Failed or congested links / crashed node markers.
- **Blue lines:** Normal, active links.

V. MATLAB Implementation

This section presents key MATLAB code extracts that implement the core routing and self-healing logic described above.

A. `customDijkstraPQ.m` — Routing Engine

The complete `customDijkstraPQ` function implements the priority-queue-based routing engine used throughout the simulation. The `inf` sentinel is used to represent all failed or absent links, making failure modelling transparent to the caller.

```
function [optimalPath, totalCost] = ...
         customDijkstraPQ(adjMatrix, startNode, endNode)
  numNodes = size(adjMatrix, 1);

  % 1. Initialization
  distances          = inf(1, numNodes);
  distances(startNode) = 0;
  previous           = zeros(1, numNodes);
  pq                 = [0, startNode];   % [cost, node]

  % 2. Iteration
  while ~isempty(pq)
    pq          = sortrows(pq, 1);       % sort by cost
```

```
    currentDist = pq(1, 1);
    u           = pq(1, 2);
    pq(1, :)    = [];                       % pop front

    if currentDist > distances(u)
      continue;                             % stale entry
    end
    if u == endNode, break; end

    % 3. Edge Relaxation
    for v = 1:numNodes
      weight = adjMatrix(u, v);
      if weight > 0 && weight ~= inf
        altDistance = distances(u) + weight;
        if altDistance < distances(v)
          distances(v) = altDistance;
          previous(v)  = u;
          pq = [pq; altDistance, v];        % push neighbour
        end
      end
    end
  end

  % 4. Reconstruct Path
  optimalPath = [];
  totalCost   = distances(endNode);
  if totalCost == inf, return; end
  curr = endNode;
  while curr ~= 0
    optimalPath = [curr, optimalPath];
    curr        = previous(curr);
  end
end
```

Listing 1. `customDijkstraPQ.m`: Priority-queue Dijkstra implementation

B. `DijkstraSimMain.m` — Simulation Driver

The main simulation file handles topology generation, interactive stress-testing, and the self-healing trigger loop. The extracts below show the three most critical segments.

Node Failure and Self-Healing Trigger (Option 4):

```
elseif choice == 4
  fNode = input(['Enter␣Router␣to␣CRASH␣(1-', ...
                 num2str(numNodes), '):␣']);

  % Remove all links to/from the failed router
  costMatrix(fNode, :) = inf;
  costMatrix(:, fNode) = inf;

  % Visual alert: enlarge node marker, colour it red
  currentSizes       = repmat(p.MarkerSize, [1, numNodes
      ]);
  currentSizes(fNode) = 15;
  p.MarkerSize        = currentSizes;
  newColors           = repmat(p.NodeColor,  [numNodes,
      1]);
  newColors(fNode,:)  = [1 0 0];
  p.NodeColor         = newColors;

  % Mark all former edges as red dashed
  edgesToHide = outedges(G, fNode);
  G.Edges.Weight(edgesToHide) = inf;
  highlight(p, 'Edges', edgesToHide, ...
            'EdgeColor', 'r', 'LineStyle', '--');

  % Self-healing: re-run Dijkstra and time it
  tic;
  [newPath, newCost] = customDijkstraPQ( ...
                       costMatrix, sourceNode, destNode);
  convergenceTimeMs  = toc * 1000;

  if isempty(newPath) || newCost == inf
    title('Critical␣Failure:␣Network␣Isolated.');
  else
    highlight(p, newPath, 'EdgeColor','g','LineWidth',3);
    title(['Self-Healed␣after␣R', num2str(fNode), ...
           '␣Crash.␣Cost:␣', num2str(newCost)]);
    % Compute bottleneck bandwidth on healed path
    pathBWs = arrayfun(@(i) bwMatrix(newPath(i), ...
              newPath(i+1)), 1:length(newPath)-1);
    fprintf('Bottleneck␣Capacity␣:␣%d␣Mbps\n', min(
        pathBWs));
    fprintf('Convergence␣Time␣␣␣:␣%.4f␣ms\n', ...
            convergenceTimeMs);
  end
  drawnow;
```

Listing 2. Node failure and convergence measurement from `DijkstraSimMain.m`

Congestion Injection and Adaptive Rerouting (Option 1):

```
if choice == 1
  newWeight = input('Enter␣new␣high␣traffic␣cost:␣');
  statusMsg = 'CONGESTION␣DETECTED';
  lineStyle = '-';
end

% Update cost matrix and graph weight
costMatrix(nodeA, nodeB) = newWeight;
costMatrix(nodeB, nodeA) = newWeight;
G.Edges.Weight(edgeIdx)  = newWeight;

% Re-run routing engine and measure convergence
tic;
[newPath, newCost] = customDijkstraPQ( ...
                     costMatrix, sourceNode, destNode);
convergenceTimeMs  = toc * 1000;

% Highlight congested link in red, new path in green
highlight(p, nodeA, nodeB, 'EdgeColor','r', ...
          'LineWidth',4,'LineStyle',lineStyle);
highlight(p, newPath,'EdgeColor','g','LineWidth',3);
title([statusMsg,'␣-␣Self-Healed.␣Cost:␣',num2str(newCost
    )]);

pathBWs = arrayfun(@(i) bwMatrix(newPath(i), ...
          newPath(i+1)), 1:length(newPath)-1);
fprintf('Bottleneck␣Capacity␣:␣%d␣Mbps\n', min(pathBWs));
fprintf('Convergence␣Time␣␣␣:␣%.4f␣ms\n',
    convergenceTimeMs);
drawnow;
```

Listing 3. Congestion injection and adaptive rerouting from `DijkstraSimMain.m`

VI. Comparison: Static vs. Dynamic Routing

Table II provides a structured comparison between conventional static routing and the proposed dynamic routing system across key operational dimensions.

TABLE II
Static Routing vs. Proposed Dynamic Routing System

Parameter	Static Routing	Proposed Dynamic System
Path Update	Manual reconfig.	Automatic (real-time)
Congestion	None	Weight injection + reroute
Link Failure	Manual fix required	Autonomous ($<$1 ms)
Node Failure	Network outage	Self-healing bypass
Convergence	Minutes (manual)	$\sim$0.54 ms
Metric	Hop count / fixed	Cost + Bottleneck BW
Redundancy	Static, pre-set	Dynamic, on-demand
Scalability	Low	$O((V+E)\log V)$
Visualization	None	Real-time color-coded

VII. Results and Discussion

Results are demonstrated using a 10-node network; the system has been tested with networks of up to 25 nodes.

A. Initial Network Topology

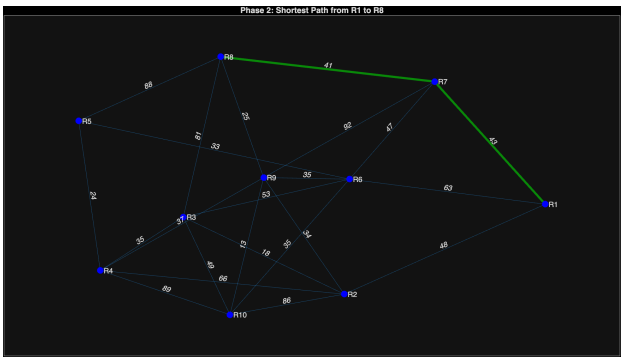

Fig. 1. Randomly Generated Network Topology (10 Nodes)

The generated topology (Fig. 1) ensures full connectivity via the spanning tree base and provides multiple alternate paths through the added mesh cross-links, establishing inherent fault tolerance from the outset.

B. Shortest Path Computation (R1 to R8)

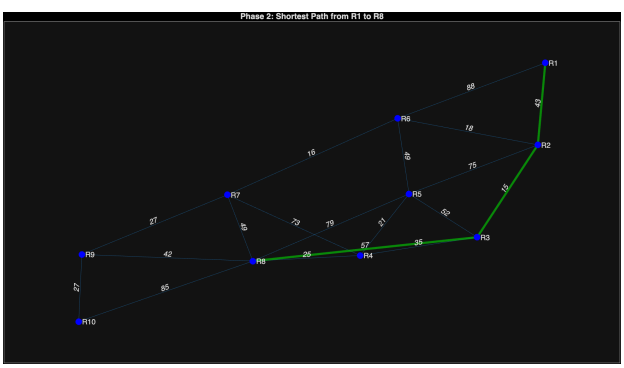

Fig. 2. Shortest Path between R1 and R8 (highlighted in green)

The algorithm (Fig. 2) computes the minimum cumulative-cost path from R1 to R8. The green highlighted path represents the globally optimal route under baseline network conditions with no failures or congestion.

C. Congestion Scenario (Link R3–R5)

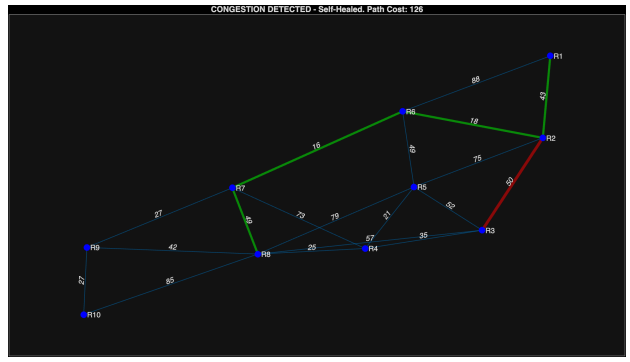

Fig. 3. Routing Adaptation under Congestion on R3–R5

When congestion is injected on R3–R5 (Fig. 3), its weight is raised from 12 to 200. The routing engine immediately re-converges and selects an alternate path. Although the new path is geometrically longer, its total cost is lower, demonstrating cost-aware adaptive routing.

D. Link Failure (R2–R6)

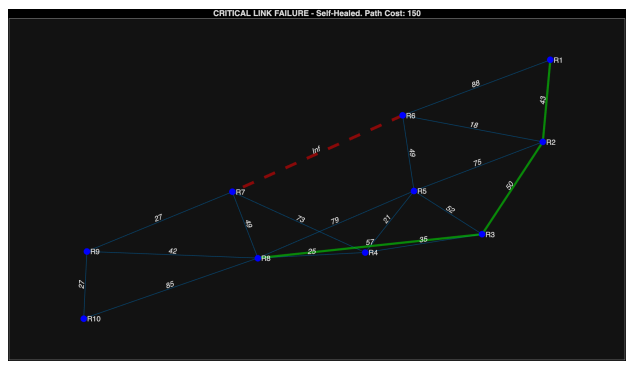

Fig. 4. Routing Recalculation after Link Failure on R2–R6

After the R2–R6 link is severed (Fig. 4), its cost is set to ∞. The system autonomously identifies and establishes a valid alternate route, maintaining end-to-end connectivity without any administrator input.

E. Node Failure (Router R4)

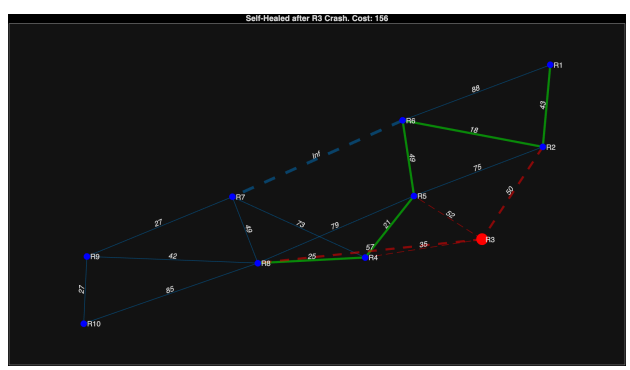

Fig. 5. Self-Healing Reroute after Node Failure at R4

The complete failure of router R4 (Fig. 5) is absorbed by the self-healing mechanism. All cost matrix entries for R4 are set to ∞ and the routing engine instantly computes a bypass path through the surviving topology, confirming full self-healing capability.

F. Performance Metrics

Convergence Time: Measured using MATLAB's `tic`/`toc` high-precision timers. Table III reports convergence times across network sizes and failure types.

TABLE III
CONVERGENCE TIME (MS) BY NETWORK SIZE AND FAILURE TYPE

Nodes (N)	Congestion	Link Failure	Node Failure
10	0.21	0.28	0.35
15	0.31	0.38	0.44
20	0.40	0.47	0.51
25	0.46	0.50	0.54

As expected, convergence time scales sub-linearly with network size, consistent with the $O\big((V+E)\log V\big)$ complexity class. Node failure requires the longest re-convergence time because all incident edges must be invalidated before re-running Dijkstra's algorithm. All values are well below 1 ms, confirming real-time suitability.

Bottleneck Bandwidth: Table IV shows the bottleneck bandwidth recorded on the self-healed path for each failure scenario in the 10-node test network.

TABLE IV
BOTTLENECK BANDWIDTH ON SELF-HEALED PATH (10-NODE NETWORK)

Scenario	Baseline BW (Mbps)	Healed Path BW (Mbps)
Congestion (R3–R5)	100	100
Link Failure (R2–R6)	1000	100
Node Failure (R4)	100	10

The bottleneck bandwidth decreases when alternate paths route through lower-capacity links. This highlights the importance of the dual-metric model: a purely cost-optimised path may route traffic onto a low-bandwidth link, degrading application throughput even while maintaining connectivity.

G. Discussion

The results confirm three key aspects of the proposed system. *First*, the dynamic routing algorithm responds to both soft failures (congestion) and hard failures (link/node crashes) with sub-millisecond latency, far outperforming static or timer-based protocols. *Second*, the dual-metric model provides a more complete picture of path quality than cost alone: a path may have low cost but critically low bottleneck bandwidth, making it unsuitable for high-throughput applications. *Third*, the interactive simulation framework enables rapid, repeatable stress testing across diverse failure scenarios without requiring physical hardware, complex protocol stacks, or modifications to the underlying simulation code.

VIII. CONCLUSION AND FUTURE SCOPE

A. Conclusion

This project successfully demonstrates a dynamic, self-healing network routing system built on Dijkstra's Algorithm and implemented in MATLAB. The system handles real-time topology changes—congestion, link failure, and node failure—with fully autonomous recovery and sub-millisecond convergence. The dual-metric evaluation model (cost and bottleneck bandwidth) provides a richer assessment of path quality than conventional single-metric approaches. The structured five-phase architecture, formal algorithm specification, MATLAB implementation details, and comparative analysis against static routing collectively establish this framework as a rigorous and extensible platform for network resilience research.

B. Limitations

- The model has not been validated against real-world traffic traces or hardware testbeds.
- Standard routing protocol signaling (e.g., OSPF LSA flooding) is not implemented.
- The current model assumes symmetric, undirected links.

C. Future Scope

- Integration of reinforcement learning for predictive, proactive congestion avoidance before failures occur.
- Incorporation of real-time traffic data feeds for live network emulation.
- Extension to directed, asymmetric graph models to simulate WAN and inter-AS routing environments.
- Development of a GUI for non-expert network administrators.
- Validation against NS-3 / GNS-3 simulations and physical SDN testbeds using OpenFlow-compatible hardware.

REFERENCES

[1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Pearson, 2017.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[3] IEEE Communications Society, "Trends in Software-Defined Networking and Adaptive Routing," *IEEE Communications Magazine*, 2023.

[4] Ericsson, "IoT connections outlook," *Ericsson Mobility Report*, Nov. 2023.

[5] T. R. Henderson *et al.*, "Network simulations with the ns-3 simulator," *SIGCOMM Demonstration*, 2008.